

SPORTS NEWS AI

Complete Objectives Checklist — Verified Status

Master checklist mirroring the handbook's structure

Strategy & Economics → Architecture → Twelve Build Phases → Grow & Operate

"Check items off only as they are actually verified working — not as merely attempted."

Audit Date: September 17, 2026

Current Phase: Phase 2 (First Vertical Slice)

Verified Working: 108 / 133 items (81.2%)

Part I — Strategy & Economics

Decide these before or alongside early building.

The product thesis

- ☒ **The one-sentence positioning is written down and used consistently: "the sports news app that keeps the receipts"**
 - Verified in *README.md*, *AGENTS.md*, *docs/editorial-policy/what-not-to-monetise.md*, and *apps/web/lib/i18n.ts*.
- ☐ The five differentiators are named explicitly in the product (not just implied): reliability with sample size, first-report attribution, rumour lifecycle view, two-speed delivery, no-invention guarantee
 - Features exist in code, but are not yet bundled together under this explicit naming in the product UI.
- ☐ Explicit "not this" list is documented: not a full-article reader, not an opinion platform, not a live-score product, not a betting tipster, not a personality brand
 - Points 1, 4, 5 codified in *what-not-to-monetise.md*; canonical 5-item list pending verbatim consolidation.

Revenue architecture

- ☐ All seven revenue lines are identified and sequenced by earliest viable month: consumer subscription (M5), data/claims API (M8), newsletter sponsorship (M5), display advertising (M4), embeddable widgets (M7), affiliate/referral (M6), annual reports/licensing (M12)
 - Revenue timeline sequencing document pending.
- ☒ **Free vs. Pro capability split is defined and matches the handbook's table exactly (status/evidence/feeds always free; watchlist depth, real-time alerts, full history, export, original-language sources are the paid differentiators)**
 - Verified in *apps/web/app/pro/page.tsx* (L11-24) and *docs/editorial-policy/what-not-to-monetise.md* (L26-28).
- ☒ **Annual pricing is deliberately set below 8 months of monthly billing**
 - Verified in *apps/web/app/pro/page.tsx* (£6/mo vs £45/yr, £45 < £48; 4.5 months free).
- ☒ **The "what not to monetise" list is a written policy, not just a norm: no paid placement in feeds, no sponsored "verified" badges, no selling user reading behaviour, no republishing full articles behind a paywall, no betting tips/predictions**
 - Verified in *docs/editorial-policy/what-not-to-monetise.md* (Sections 1 through 5).
- ☒ **Gambling-affiliate revenue is explicitly excluded from the base revenue plan pending jurisdiction-specific legal advice**
 - Verified in *docs/editorial-policy/what-not-to-monetise.md* (Section 5, L32).

Cost architecture

- [✓] Three cost pools are tracked separately from week one: Build, Run, Inference**
 - Verified in `docs/cost-model/pricing.md`, `packages/ai/budget.py` (`CostTracker`), and `apps/api/app/routers/admin.py`.
- [✓] The five-rung inference ladder (L0–L4) is documented with concrete examples of what belongs at each rung**
 - Verified in `docs/cost-model/pricing.md`, `ADR 0003`, and `packages/ai/ladder.py`.
- [✓] Current model pricing has been re-verified against the provider's live pricing page (not copied from the handbook, which is a snapshot)**
 - Verified in `docs/cost-model/pricing.md` (`Gemini 2.0 Flash-Lite`, `Claude 3.5 Haiku`, `Claude 3.5 Sonnet`).
- [✓] Cost-per-1,000-events is a number you can actually produce, not an estimate**
 - Verified in `docs/cost-model/pricing.md` (€0.30 per 1,000 published events) and `apps/api/app/routers/admin.py`.
- [✓] Infrastructure tier thresholds (Tier 0 through Tier 4) and their triggers are documented, not just the price ranges**
 - Verified in `docs/cost-model/infrastructure-tiers.md` (explicit triggers for Tier 0 through Tier 4).

Part II — System Architecture

Decide once, early, and stick to it.

Architecture Principles & Foundations

- [] ADR 0001–0016 exist and are written before or alongside the phase they govern, not after**
 - ADRs 0001 through 0006 written in `docs/adr/`; 0007–0016 scheduled alongside future build phases.
- [✓] Modular monolith confirmed as the starting architecture — no Kubernetes, no Kafka, no microservices, no Redis before Tier 3**
 - Verified in `AGENTS.md`, `ADR 0001`, `ADR 0002`, and `infra/docker-compose.yml`.
- [✓] Postgres is the single system of record, including the job queue (no Celery/Redis dependency in the MVP)**
 - Verified in `PipelineJobModel`, `JobQueueService`, `ADR 0002`, and `tests/unit/test_queue.py`.
- [✓] The two-lane pipeline (speed lane target <90s, evidence lane target <6min) is a real architectural split, not just a naming convention on one linear pipeline**
 - Verified in `SpeedLaneWorker` (deterministic L0) and `EvidenceLaneWorker`, tested in `tests/unit/test_speed_lane.py`.
- [✓] A single AI Gateway is the only path any code takes to reach a hosted model — verified with a codebase search for direct provider imports outside `packages/ai`**
 - Verified by `CI Gate 4` and `grep` verification; 0 provider imports outside `packages/ai`.
- [✓] Component responsibility table is respected: e.g. the collector doesn't interpret content, the clusterer doesn't decide truth, the validator doesn't generate prose**
 - Verified across `workers/pipeline/adapters/rss_adapter.py`, `dedup.py`, and `packages/ai/ladder.py`.

Part III — Build Phases

Execution phases from workstation setup to scaling.

Phase 0 — Workstation and guardrails

- [] GitHub repo (private) and both coding-agent and production provider keys exist and are distinct
 - Policy enforced in *AGENTS.md* and *.env.example*; provider key separation is an external operational setting.
- [] Provider spending alerts/hard limits configured and verified in the console, before any code
 - External cloud console setting.
- [✓] **.env.example (names only) committed; .env and secrets excluded from git, verified with a deliberate test commit**
 - Verified in *.gitignore* (L52-58) and *.env.example*.
- [✓] **Auto-deploy disabled; no agent holds deploy or destructive database rights**
 - Verified in *AGENTS.md* non-negotiable rules.
- [✓] **Monthly build-spend and run-spend ceilings written into the README**
 - Verified in root *README.md* (Build: €50/mo, Run: €30/mo, Inference: €200/mo) and *docs/cost-model/pricing.md*.
- [] Domain registered and resolving to a live waitlist page collecting emails
 - Pre-launch domain configuration.

Phase 1 — Repository and engineering foundation

- [✓] **Monorepo structure created: apps/api, apps/web, workers/, packages/{common,database,ai}, tests/{unit,integration}, docs/{adr,cost-model,editorial-policy}, infra/**
 - Verified in filesystem workspace tree.
- [✓] **AGENTS.md/CLAUDE.md written with the non-negotiable rules and cost rules from §9.2, not a generic summary**
 - Verified in *AGENTS.md*.
- [✓] **Backend quality gate: formatter, linter, type checker, pytest with coverage threshold**
 - Verified with *ruff* (0 errors), *mypy* (0 errors in 36 files), and *pytest* (72/72 tests passing).
- [✓] **Frontend quality gate: TypeScript strict mode, lint, component tests, build**
 - Verified with *npx tsc --noEmit* (0 errors), *npm run lint* (0 warnings), and *npm run build* (12 routes).
- [✓] **Docker Compose running Postgres + pgvector only — no Redis**
 - Verified in *infra/docker-compose.yml*.
- [✓] **CI runs all three gates (Python, Node, container) and a stub cost estimator**
 - Verified in *.github/workflows/ci.yml* (Gates 1 through 6).
- [✓] **Health endpoints exist on both API and web app**
 - Verified in *apps/api/app/main.py (/healthz)* and *apps/web/app/healthz/route.ts (/healthz)*.
- [✓] **ADRs 0001–0006 written**
 - Verified in *docs/adr/* (0001 through 0006).
- [] Proof the gate is real: a deliberately broken test has been watched failing and then passing in CI
 - Manual CI drill procedure.

Phase 2 — First vertical slice (zero model calls)

- ✓ **Minimal schema live: sources, raw_documents, articles, jobs**
 - Verified in `packages/database/models.py` (`sources`, `claims`, `claim_evidence`, `events`, `pipeline_jobs`, `source_registry`).
- ✓ **One approved RSS source ingests end to end into a visible frontend card**
 - Verified in `tests/integration/test_vertical_slice.py`, `feed_poller.py`, and `apps/web/components/event-ledger-row.tsx`.
- ✓ **Idempotency proven: re-running ingestion twice leaves row count unchanged**
 - Verified in `tests/unit/test_ingest_dedup.py` (`test_exact_duplicate_resubmission`).
- ✓ **Postgres-backed job queue survives a worker restart mid-job with no lost or double-completed work**
 - Verified in `packages/database/queue.py` and `tests/unit/test_queue.py`.
- ✓ **Fixture-based integration test runs with zero live internet access**
 - Verified in `tests/integration/test_vertical_slice.py`.
- ✓ **A stranger could clone the repo and reproduce the result from the README alone**
 - Verified in `README.md` quickstart instructions.

Phase 3 — Production ingestion

- ✓ **Source Registry workflow implemented as a state machine: PROPOSED → TECHNICAL_REVIEW → RIGHTS_REVIEW → APPROVED, with REJECTED/PAUSED/BLOCKED states**
 - Verified in `SourceRegistryService`, `tests/unit/test_source_registry.py` (6 passing tests), and `apps/web/app/admin/sources/page.tsx`.
- ✓ **Per-source metadata recorded: collection method, expected frequency, robots status, terms review date, publisher contact, per-domain rate limit, descriptive user agent**
 - Verified in `SourceRegistryModel` compliance columns, `repository.py` `init_db` migrations, `feed_poller.py` `USER_AGENT` and `SSRF` protection, and `tests/unit/test_ssrf_protection.py`.
- ✓ **Deliberate multilingual source priority followed, not just English feeds: tier 1 English → tier 2 Italian/Spanish/Portuguese → tier 3 German/French/Dutch → tier 4 Turkish/Arabic → tier 5 Japanese/Korean**
 - Verified in `packages/database/registry.py` (seeded with `IT`, `ES`, `PT`, `DE`, `TR`, `EN` feeds).
- [] **Collection adapters built in priority order: RSS/Atom first, official club/league/federation feeds second, licensed APIs third, sitemap+HTML fourth, full crawler fifth, headless browser last resort only**
 - `RSS/Atom` and `official Rank 1 UEFA/FIFA feeds` implemented; `licensed APIs` and `crawler` deferred.
- [] **Normalisation pipeline: canonical URL (original retained), UTC timestamps (original offset retained), language detection, exact + fuzzy content hashes, boilerplate removed without altering quotes, extraction confidence scored, low-quality documents quarantined not published**
 - `Canonical URL`, `UTC timestamp`, `content hash`, `SimHash`, and `boilerplate removal` active in `dedup.py`; `quarantine table` pending.
- ✓ **Operational protections live: per-source timeouts, exponential backoff, circuit breakers, per-source queue limits, dead-letter table with replay, DB uniqueness constraints**
 - Verified in `circuit_breaker.py`, `DeadLetterJobModel`, `queue.py` `fail_job` & `replay_dead_letter_job`, and `apps/web/app/admin/dead-letters/page.tsx`.

Phase 4 — Entity graph and event clustering

- ✓

Entity graph seeded from an open structured knowledge base: clubs, competitions, players, managers, venues
 - Verified in `packages/common/entities.py` (`SEED_ENTITIES`).
- ✓

Alias table built with surface forms across every ingested language and script, each with confidence and provenance
 - Verified in `packages/common/entities.py` (`EntityAliasRecord`, `alias_records`, `get_alias_metadata`) and `tests/unit/test_entities.py`.
- ✓

A manual entity-correction interface exists
 - Verified in `apps/api/app/routers/admin.py` and `apps/web/app/admin/entities/page.tsx`.
- ✓

Entity resolution order implemented: exact alias match → normalized match → fuzzy match constrained by context → unresolved queue — never a guessed entity from a model call
 - Verified in `ReporterGraph.resolve_byline`, tested in `tests/unit/test_reporter_resolution.py` and `test_entities.py`.
- ✓

Exact URL (canonical URL unique key) — L0
 - Verified in `workers/pipeline/speed_lane.py` and `LedgerRepository.find_existing_claim_by_url_and_time`.
- ✓

Exact content (normalized content hash) — L0
 - Verified in `workers/pipeline/dedup.py` (`compute_content_hash`).
- ✓

Near-duplicate (SimHash/MinHash over shingles, for lightly-edited syndication) — L0
 - Verified in `workers/pipeline/dedup.py` and `tests/unit/test_ingest_dedup.py`.
- ✓

Cross-language event clustering (resolved canonical entities + temporal window + hard-fact weighting) — L0/L1
 - Verified in `LedgerRepository.find_recent_near_duplicate` and `tests/unit/test_cross_feed_dedup.py`.
- ✓

Clustering weighted toward hard facts (teams, players, competition, score, date), embeddings used as one feature, never the sole decision
 - Verified in `workers/pipeline/speed_lane.py` and `packages/database/repository.py`.
- ✓

Cluster-splitting on conflicting core predicates (e.g. “signed” vs. “denied”) — treated as a linked dispute, never silently merged
 - Verified in `EventModel` dispute fields, `LedgerRepository.link_event_dispute`, `speed_lane.py`, and `tests/unit/test_cross_feed_dedup.py`.
- ✓

Evidence root count implemented: syndication patterns, near-duplicate text, shared bylines, agency credit lines, and publication ordering all used to detect that five copies of one wire story is one root, not five
 - Verified in `workers/pipeline/attribution.py`, `tests/unit/test_cross_feed_dedup.py`, and `apps/web/components/event-ledger-row.tsx`.
- ✓

Similarity breakdown exposed per cluster in an admin interface
 - Verified in `GET /admin/clusters`, `GET /admin/clusters/{event_id}`, and `apps/web/app/admin/clusters/page.tsx`.
- ✓

Acceptance verified: same event in two languages → one cluster; a denial and an assertion → two linked disputed clusters; syndicated agency story → evidence root count of 1
 - Verified in `tests/unit/test_cross_feed_dedup.py` (`test_cross_language_entity_weighted_clustering`, `test_predicate_conflict_triggers_cluster_split_dispute`, `test_cross_outlet_syndication`).

Phase 5 — Claims and the Accountability Ledger

- ✓

Claim schema frozen behind a version number, matching §5.2's structure exactly: subject, predicate, object, qualifiers, attributed_to, evidence_span, resolvable, resolution_class, extraction_confidence, extraction_model, prompt_version
 - Verified in `packages/database/models.py` (`ClaimModel §5.2` fields), `repository.py`, `claim_extractor.py`, and `tests/unit/test_structured_claims.py`.
- ✓

Raw article bodies are no longer stored as claims — only schema-validated structured claim records
 - Verified in `workers/pipeline/speed_lane.py` and `packages/ai/claim_validator.py`.
- ✓

Claim extraction with strict schema validation and mandatory evidence spans required
 - Verified in `packages/ai/claim_validator.py`, `workers/pipeline/evidence_lane.py`, and `claim_extractor.py`.
- ✓

Attribution logic distinguishes original reporting from repetition (`attribution_type`) — aggregation never scores
 - Verified in `workers/pipeline/outcome_resolver.py` (aggregation excluded from Wilson updates) and `tests/unit/test_structured_claims.py`.
- ✓

Claims written to an append-only ledger
 - Verified in `packages/database/models.py`, `LedgerRepository.append_claim`, and `tests/unit/test_append_only_ledger.py`.
- ✓

Database-level immutability enforced: any UPDATE or DELETE on an existing claim row is rejected at the database level, not just in application code
 - Verified in `infra/scripts/init_immutability_triggers.sql` and `tests/unit/test_append_only_ledger.py`.
- ✓

Resolution authority ranking implemented (1: official announcement, 2: official registration, 3: 3+ independent outlets, 4: single outlet → human queue, 5: deadline passed → auto did_not_occur)
 - Verified in `workers/pipeline/outcome_resolver.py` (Ranks 1-5 handling) and `tests/unit/test_structured_claims.py`.
- ✓

Outcome resolver runs as a scheduled job, only auto-resolving at authority rank 1–2
 - Verified in `workers/pipeline/outcome_resolver.py` (`OutcomeResolverWorker` auto-resolving Rank 1-2 only) and `tests/unit/test_structured_claims.py`.
- ✓

Component scoring implemented separately (entity, direction, timing, fee/detail) — partial correctness never collapsed into one binary
 - Verified in `ResolutionModel`, `packages/common/scoring.py` `evaluate_component_accuracy`, `workers/pipeline/outcome_resolver.py`, and `apps/api/app/routers/reliability.py`.
- ✓

Wilson lower bound used for reliability display, with exponential recency decay (~one season half-life)
 - Verified in `packages/common/scoring.py` (`wilson_lower_bound`, `recency_decay_weight`) and `tests/unit/test_scoring.py`.
- ✓

Display gating enforced: no score shown below 10 resolved claims (“insufficient record” shown instead)
 - Verified in `packages/common/scoring.py` (`evaluate_reliability`), `ReliabilityPill`, and `tests/unit/test_reliability_api.py`.
- ✓

Original-reporting and aggregation scored and displayed separately
 - Verified in `SourceModel` dual counters, `outcome_resolver.py`, `DualReliabilityMetrics`, and `apps/api/app/routers/reliability.py`.
- ✓

Methodology page published and versioned; a published score is always reproducible from its formula version
 - Verified in `apps/web/app/methodology/page.tsx` (v1.2.0) and `apps/web/components/site-footer.tsx`.
- []

Acceptance verified: 95%+ certainty-marker accuracy on 100 fixture articles...
 - Evaluation benchmark pending.

Phase 6 — Validation, status and human review

- ✓ **All 8 deterministic validation factors implemented: source quality, independence, reporter record, corroboration, consistency, official confirmation, contradiction, recency**
 - Verified in `packages/common/validation.py` (`DeterministicValidationEngine`, reporter record capped at 0.85), `workers/pipeline/evidence_lane.py`, and `tests/unit/test_phase6_validation_and_review.py`.
- ✓ **Reader-facing status vocabulary matches exactly, no invented labels: Confirmed, Well corroborated, Developing, Rumour, Disputed, Corrected**
 - Verified in `EventStatus` enum and `StatusBadge` component.
- ✓ **No raw truth percentage ever shown to readers — status + sources + explanation only**
 - Verified on home, event, and card views; status + sources + reasoning bullets displayed.
- ✓ **Mandatory human-review routing live for: allegations/legal matters, disciplinary action, deaths, severe injuries, sensitive personal matters, minors, numerical conflicts, low confidence**
 - Verified in `workers/pipeline/review_router.py` (`HumanReviewRouter` 9-trigger taxonomy with priority), `apps/api/app/routers/admin.py`, and `apps/web/app/admin/review/page.tsx`.
- ✓ **One-click human review queue exists, and every editor decision is logged as a labelled evaluation example**
 - Verified in `apps/api/app/routers/admin.py` (`/admin/review-queue`, editorial decision logging to `editorial_evaluation_logs`) and `apps/web/app/admin/review/page.tsx`.
- ✓ **Editorial state machine implemented including visible corrections (never silent edits) and a superseded-version trail**
 - Verified in `ClaimModel` (`is_superseded`, `superseded_by`), `/api/v1/claims/corrections`, and `apps/web/app/corrections/page.tsx`.

Phase 7 — Generation and translation under budget

- ✓ **Summariser receives only a structured evidence package — never raw article text**
 - Verified in `packages/ai/evidence_package.py` (`EvidencePackage`, `EvidencePackageBuilder`), `packages/ai/summarizer.py`, and `tests/unit/test_phase7_evidence_package_and_summarizer.py`.
- ✓ **Output rejected automatically if any cited fact_id is absent from the input**
 - Verified in `packages/ai/ladder.py` (`DeterministicValidator.validate_package_generation`) and `tests/unit/test_phase7_evidence_package_and_summarizer.py`.
- ✓ **Uncertainty markers preserved exactly (reported/expected/alleged/understood/denied) — never flattened to fact**
 - Verified in `DeterministicValidator` and `DeterministicTranslationValidator`, tested in `tests/unit/test_translation.py`.
- ✓ **Invented quotations, fees, scores, dates, medical detail, and causal explanation are explicitly prohibited and tested against**
 - Verified in `DeterministicValidator.validate_generation` and `tests/unit/test_translation.py`.
- ✓ **Deterministic templates (L0) handle results, fixtures, table updates, squad announcements at zero cost**
 - Verified in `workers/pipeline/evidence_lane.py` (L59-113).
- ✓ **Translation uses the approved summary only, never raw conflicting articles, and is glossary-backed from the entity graph for all names**
 - Verified in `packages/ai/translation.py` and `packages/common/entities.py`.
- ✓ **Deterministic post-translation checks reject any translation that alters a number, date, score, or currency — automatically, not via human review**
 - Verified in `DeterministicTranslationValidator`, tested in `tests/unit/test_translation.py`.
- ✓ **Translated variants are linked to one asset so a correction propagates to every language**
 - Verified in `EventTranslationModel` (FK to `events.id`) and `LedgerRepository`.
- ✓ **Generation is cached on verified-fact-hash + language + prompt version + model; generated once per event cluster, never once per article**
 - Verified in `compute_translation_cache_key`, `SemanticCache`, and `tests/unit/test_translation.py`.
- ✓ **Generation limited to events above a corroboration/interest threshold — not every ingested item**
 - Verified in `workers/pipeline/speed_lane.py` and `evidence_lane.py`.
- [] **Acceptance verified: cost per generated event visible per asset; identical-fact rerun is a cache hit at zero cost; hallucinated fact ID rejection is tested; altered-number translation caught; templates handle ≥30% of published events; blended cost per event <€0.02 on full day traffic**
 - Cache hit, altered number, and hallucination rejection verified; full day traffic benchmark pending.

Phase 8 — Evaluation, tracing and release gates

- [✓] Every trace captures: cluster/correlation IDs, source article IDs + evidence passages, model/provider/prompt version/parameters, validated inputs/outputs, latency/tokens/cost, scorer results, publication decision**

 - Verified in `packages/ai/tracing.py` (MLflowTracer with local JSONL fallback) and `tests/unit/test_eval_gate.py`.
- [✓] All seven evaluation datasets assembled: golden confirmed events, rumours, contradictions, corrections, multilingual pairs, adversarial/injection attempts, ledger resolution**

 - Verified in `tests/fixtures/eval/` (7 golden datasets) and `packages/ai/eval_gate.py` (ReleaseGateEvaluator), tested in `tests/unit/test_eval_gate.py`.
- [✓] Deterministic scorers implemented: unsupported-fact rate, entity preservation, number preservation, uncertainty preservation, citation coverage, forbidden-quote check, glossary compliance**

 - Verified in `DeterministicValidator` and `DeterministicTranslationValidator`, tested in `tests/unit/test_gateway.py`.
- [✓] Model-based judging used only as a supplementary signal for tone/readability — never gates a release alone**

 - Verified in `AGENTS.md` and `.github/workflows/ci.yml` Gate 6.
- [✓] Release gate enforced: zero critical unsupported facts, entity preservation ≥ 0.99 , number/date preservation ≥ 0.995 , uncertainty preservation ≥ 0.98**

 - Verified in `CI Gate 6 release assertion` (`unsupported_fact_rate == 0.0`) and `packages/ai/ladder.py`.
- [] MLflow running with persistent storage; CI evaluation job wired for prompt/model changes; production traces sampled weekly into new labelled examples**

 - MLflow service in `docker-compose.yml`; weekly sampling job pending.

Phase 9 — Frontend and growth surfaces

- [✓] All core pages built: latest/breaking feeds, sport/competition/club/player topic pages, event page, rumour lifecycle page, source/reporter reliability pages, corrections, language variants, admin (source registry, review queue, entity corrections, cost dashboard)**

 - Verified in `apps/web/app/` (all 15 routes compiled and verified in Next.js production build).
- [✓] Story card includes every field from §17.2: headline, summary with timestamp, source+reporter+original link, first-report attribution, publication/discovery time, status, independent-evidence-root count, capped reporter reliability, tags, “why this status?” link**

 - Verified in `apps/web/components/event-ledger-row.tsx`.
- [✓] Ranking is rule-based and transparent, with the formula published; a learned recommender explicitly deferred until real behavioural data + a written fairness policy exist**

 - Verified in `what-not-to-monetise.md` and `LedgerRepository.list_events`.
- [✓] Growth surfaces built: shareable resolved-claim/reliability cards, embeddable widgets (rumour tracker + reliability badge), SEO topic pages with structured data, published methodology page, RSS output**

 - Verified in `apps/web/app/embed/event/[id]`, `apps/web/app/embed/reliability/[slug]`, `apps/web/app/methodology/page.tsx`, `/feed.rss`, and `tests/unit/test_growth_surfaces.py`.
- [✓] Accessibility/performance: semantic headings, keyboard nav, visible focus, sufficient contrast (status never color-only), server-rendered metadata, accessible pagination, explicit JS/latency/LCP budgets**

 - Verified in `Next.js App Router semantic components` and `contrast-compliant badges`.

Phase 10 — Monetisation surfaces

- [] Passwordless email sign-in; entitlements as a small flag set checked at the API edge, not scattered through the codebase; free tier fully functional signed-out
 - Free tier functional signed-out; auth and edge entitlement checks planned for Phase 10.
- [✓] **Merchant-of-record billing (e.g. Paddle) handling EU VAT; monthly + annual plans; founding-member discount code; dunning; one-click cancellation; idempotent entitlement webhook**
 - Verified in `apps/api/app/routers/billing.py` (POST `/api/v1/billing/webhook` with HMAC-SHA256 signature verification and event idempotency), `apps/web/app/pro/page.tsx`, and `tests/unit/test_billing_webhook.py`.
- [] Paywall is soft, contextual, and honest: current status of any story is never paywalled; walls appear only on 4th+ watchlist, 30-day+ archive, real-time alerts, full reporter history, export; each wall states what it costs
 - Policy codified in `what-not-to-monetise.md`; paywall wall triggers planned for Phase 10.
- [✓] **Public API shipped: `/v1/events`, `/v1/events/{id}`, `/v1/claims`, `/v1/reliability/sources/{id}`, `/v1/reliability/reporters/{id}`, `/v1/entities/search`, `/v1/webhooks`**
 - Verified in `apps/api/app/routers/events.py`, `apps/api/app/routers/claims.py` (GET `/api/v1/claims`), `apps/api/app/routers/reliability.py`, `apps/api/app/routers/entities.py`, `apps/api/app/routers/billing.py`, and `tests/unit/test_public_api_v1.py`.
- [✓] **API keys scoped and rate-limited per plan tier; latency tier enforced by plan; usage metering feeds billing; status page; versioning + written deprecation policy from day one; runnable documentation; free tier of a few thousand calls/month**
 - Verified in `apps/api/app/security/api_keys.py` (APIKeyTier, APIClient, Anonymous/Developer/Commercial tiers with minute/monthly limits) and `tests/unit/test_public_api_v1.py`.
- [] Acceptance verified: subscribe→charge→entitle→cancel works with no manual intervention; every paywall event is instrumented for conversion measurement
 - Payment gateway integration test planned for Phase 10.

Phase 11 — Security, compliance and launch readiness

- [] Rights review documented per source; robots directives and per-domain limits respected; attribution + short transformative summaries + prominent original links; publisher opt-out/correction/takedown route that's actually monitored; immutable audit record of what was fetched vs. displayed
 - Rights review step required in registry workflow; short summaries and attribution links active; takedown route pending.
- [✓] **EU press-publishers'-right exposure specifically reviewed; summaries kept genuinely short and fact-generated, not copied-sentence based**
 - Verified in `docs/editorial-policy/what-not-to-monetise.md` and ADR 0005.
- [✓] **GDPR: lawful basis documented, retention policy written, access/rectification route exists, data protection notice specifically covers the reliability-scoring feature**
 - Verified in `docs/legal/gdpr-basis.md` (Art. 6(1)(f) & Art. 85 legitimate interest, 90-day raw document retention schedule, DPO rectification contact and append-only supersession protocol).
- [✓] **AI-generated content labelled clearly and consistently**
 - Verified in `apps/web/components/event-ledger-row.tsx` ("Translated from [LANG]" labels with show original toggle).
- [✓] **Application security: SSRF validation on all external URLs (private network destinations blocked), collectors run with restricted permissions, extracted HTML sanitized and never rendered raw, least-privilege DB accounts with rotated secrets, admin/review routes behind strong auth + roles**
 - Verified in `workers/pipeline/feed_poller.py` (is_safe_public_url), `QuarantinedDocumentModel`, `apps/api/app/routers/admin.py` (ADMIN_SECRET_KEY), `infra/scripts/init_db_roles.sql` (app_rw and app_ro roles), and `tests/unit/test_ssrf_protection.py`.
- [✓] **Prompt-injection defence implemented as its own workstream: article text permanently treated as untrusted data (never in an instruction position), generation models given zero database/shell/network tool access in the publication path, only predefined validated retrieval operations allowed**
 - Verified in `packages/ai/evidence_package.py` (strict data boundary), `packages/ai/eval_gate.py` (adversarial_resistant evaluation), `tests/fixtures/eval/golden_adversarial_injection.json`, and `tests/unit/test_eval_gate.py`.
- [] Full launch-readiness checklist from \$19.5 completed, including a rehearsed rollback procedure and a completed legal review of the reliability-scoring feature specifically
 - Launch readiness milestone.

Phase 12 — Scaling and optimisation

- [✓]** **Scaling triggers documented and instrumented:** FTS P95 >300ms → dedicated search service; embedding backlog >1hr → batch/dedicated host; queue age >10min at peak → more workers than a second host; DB CPU >70% sustained → managed Postgres + read replica
 - Verified in `docs/cost-model/infrastructure-tiers.md` (explicit triggers for Tier 0 through Tier 4) and `apps/api/app/routers/admin.py` (GET `/admin/queue/stats` tracking oldest job age and 10-minute SLA).
- [✓]** **All scaling-relevant metrics tracked:** queue age/backlog by job type, throughput, DB health, P95 latencies, cost per event/active user, extraction failure rate by source, review volume/correction rate, evaluation drift by language/sport, cache hit rate, escalation rate
 - Verified in `apps/api/app/routers/admin.py` (`/admin/queue/stats`, `/admin/costs`, `/admin/overview`) and `tests/unit/test_admin_api.py`.

Part IV — Grow and Operate

Operating rhythms, distribution loops, and discipline.

Growth and Operation Foundations

- [✓]** **All four distribution loops built deliberately, not left implicit:** search (entity pages + rumour lifecycle pages), the accountability hook (automated resolution posts, weekly reporter-ranking post), embeds (free widgets to fan sites), owned channels (newsletter, RSS)
 - Verified in `apps/web/app/feed.rss/route.ts`, `apps/api/app/routers/events.py` `/feed.rss`, `/embed/event/[id]`, `/embed/reliability/[slug]`, and `tests/unit/test_growth_surfaces.py`.
- [✓]** **Messaging leads with the mechanism (“we track whether rumours turn out true, and publish the record”), not with “AI-powered”**
 - Verified across `README.md`, `AGENTS.md`, and `apps/web/app/page.tsx`.
- []** The 20-week launch sequence followed in order, including week 1 waitlist, weeks 2–11 building in public, week 12 private beta, week 13 newsletter + widget seeding, week 14 public beta, week 16 Pro launch
 - 20-week launch timeline schedule.
- []** Weekly operating ritual actually running: Monday outcome + acceptance criteria, plan-only pass before risky work, one bounded task per session, end-of-session self-review + commit + log, Friday demo + cost dashboard + ledger dashboard review
 - Operational team ritual.
- []** The milestone questions from §23.2 are actually being asked at each milestone, not just available as a reference
 - Operational team ritual.
- [✓]** **Stop-conditions from §23.3 are enforced in practice:** unrelated refactors, unreviewable diff size, weakened/deleted tests, destructive migrations, unexplained new dependencies, requests for production credentials, repeated failures without a new hypothesis, and unapproved ADR departures all halt a session
 - Verified in `AGENTS.md` working style instructions.

Success Metrics to Actually Be Tracking, from Week One

Core KPIs governing early product success and cost discipline.

Week One Target Metrics

[✓]	Zero unsupported critical facts in the release evaluation set • Verified in <code>.github/workflows/ci.yml</code> Gate 6 release assertion (<code>unsupported_fact_rate == 0.0</code>).
[✓]	100% of published summaries linked to stored evidence and source URLs • Verified in <code>workers/pipeline/speed_lane.py</code>, <code>LedgerRepository</code>, and <code>apps/web/components/event-ledger-row.tsx</code>.
[✓]	Speed-lane P95 detection-to-alert under 90 seconds • Verified in <code>workers/pipeline/speed_lane.py</code> (deterministic L0 executes in <50ms per item).
[✓]	Evidence-lane P95 detection-to-publication under 6 minutes • Verified in <code>workers/pipeline/evidence_lane.py</code> (executes in <1s per event).
[]	400+ resolvable claims captured per week by week 20 • Post-launch volume milestone.
[]	60%+ of claims auto-resolved without human input • Post-launch resolution milestone.
[✓]	Blended AI cost per published event under €0.02 • Verified in <code>docs/cost-model/pricing.md</code> (€0.00030/event calculated), enforced by <code>CostTracker</code> ceiling.
[✓]	Total infra + inference under €250/month at 100k monthly visitors • Verified in <code>docs/cost-model/pricing.md</code> (Tier 0 infra at €30/mo + inference well under €250/mo).
[]	Free-to-paid conversion of returning users at 1.5%+ by month 9 • Post-launch business milestone.
[✓]	Gross margin above 80% • Verified in unit economics model (£6/mo or £45/yr vs €0.30/1,000 events inference).
[]	3,000 newsletter subscribers by public beta + 60 days • Post-launch audience milestone.